

Manual de Proyecto

ARGENTUM ONLINE

Taller de Programación — Cátedra Veiga — 1er cuatrimestre 2026

Aylén Bartomeo	— 111738
Franco Bustos	— 110759
Alexander Coronado	— 111256
Bruno Pezman	— 110457

Índice

1. Introduccion	2
2. Distribución del Trabajo e Integrantes	2
2.1. Aylén Bartomeo	2
2.2. Alexander Coronado	3
2.3. Franco Bustos	5
2.4. Bruno Pezman	11
3. Herramientas Utilizadas y Fuentes de Aprendizaje	13
4. Estado final de Entrega y Retrospectiva	13

1. Introduccion

En esta sección se detalla el rol principal de cada integrante del equipo y los módulos específicos en los que trabajó activamente durante el ciclo de vida del proyecto.

2. Distribución del Trabajo e Integrantes

2.1. Aylén Bartomeo

Trabajó principalmente en el **cliente del juego** y en el **editor de mapas**, ambos desarrollados en **C++ con SDL2**, de los que desarrolló buena parte de sus vistas y funcionalidades. El desarrollo fue colaborativo —algunas piezas las implementaron otros integrantes del equipo— y, en lo que le tocó, priorizó una arquitectura modular: unidades pequeñas, separadas por responsabilidad y testeables, manteniendo desacoplados los distintos módulos.

Cliente gráfico

Vistas / pantallas:

- **Mundo de juego:** vista principal, donde se renderiza el mapa y la acción en tiempo real.
- **HUD estilo Argentum Online:** barras de vida, maná y experiencia, inventario con íconos y equipamiento.

Funcionalidades:

- **Integración del cliente con el protocolo:** conexión al servidor mediante hilos de recepción y envío y colas *thread-safe*; el cliente consume los *snapshots* del mundo y envía los comandos del jugador.
- **Render del mundo:** dibujo del terreno multicapa (suelo, decoración y techos), de los ítems en el piso y el oro, y de las entidades —jugadores, ciudadanos (comerciante, banquero, sacerdote), monstruos y *bosses*—, cada una con su sprite según tipo, raza y dirección. La cámara sigue al jugador.
- **Movimiento:** animación de caminata direccional por frames y movimiento suavizado (interpolación de la posición entre *snapshots*).
- **Inventario, equipamiento y barra de vida:** renderizado de los ítems en el inventario y en el equipamiento del panel (íconos reales y equipar con doble clic) y de la barra de vida sobre las entidades dañadas.
- **Bucle a tasa constante** (*ConstantRateLoop*) para un ritmo de juego estable.

Editor de mapas

Editor **multipantalla** que produce el mismo formato JSON de mapa que consumen el servidor y el cliente (es decir, el contrato entre los tres).

Vistas / pantallas:

- **Pantalla principal** con el canvas del mapa.
- Pantallas dedicadas para **Terrenos**, **Ítems**, **Monstruos** y **Ciudadanos**, cada una con su paleta.
- **Selector de mapas** para elegir o crear un mapa en vivo, y un **diálogo** para ingresar la cantidad de oro al colocarlo.

Funcionalidades:

- **Canvas con render del mapa** (*render-to-texture*), con *zoom* (+/-) y desplazamiento (*pan*) con las flechas.
- **Estampado de bloques de terreno** como *prefabs*: ciudad, bosque, mazmorra, desierto y playa. Cada bloque incluye su propia decoración, colisiones, edificios/NPCs y zonas (por ejemplo, la ciudad con iglesia, banco y comercio; los desiertos con sus cactus, la playa con sus palmeras, etc.).
- **Colocación de entidades** desde paletas: ítems, monstruos y ciudadanos, con validaciones por zona (p. ej. arañas y goblins solo en bosque, esqueletos y golems en desierto, ciudadanos restringidos a los edificios de la ciudad, nada en la playa, monstruos vedados en ciudad y mazmorra).
- **Goma inteligente**: borra “lo de más arriba” en una celda, en orden monstruo → ciudadano → ítem → bloque de terreno.
- **Guardar, abrir y crear mapas** desde el propio editor, escribiendo el JSON.

2.2. Alexander Coronado

Trabajó principalmente en la **arquitectura de red**, **mecánicas de combate a distancia** y la **representación visual** del sistema. La implementación, distribuida entre el **cliente** y el **servidor**, se enfocó en mantener un alto grado de desacoplamiento aplicando patrones de diseño para aislar la lógica de negocio, la persistencia de datos y el motor de renderizado.

Sistema de clanes

Arquitectura *backend* orientada a servicios para la gestión de agrupaciones de jugadores. Se implementó una separación de responsabilidades estricta mediante los patrones de diseño *Service* y *Repository*, aislando la lógica de negocio de la persistencia de datos y el enrutamiento de comandos de red.

Funcionalidades:

- **Gestión del ciclo de vida**: funcionalidades para la creación, disolución, peticiones de unión y modificación del estado del clan encapsuladas en las clases **Clan** y **ClanService**.
- **Persistencia abstraída**: interfaz de acceso a datos mediante **ClanRepository**, permitiendo la carga y guardado del estado de los clanes sin acoplar el modelo de dominio a la capa de infraestructura.

- **Controlador de peticiones:** procesamiento y delegación de solicitudes de red asociadas a clanes a través del `ClanController`, validando permisos y roles de usuario (fundador vs. miembro).
- **Acoplamiento controlado:** integración con el estado del servidor mediante `IWorldContext` para interacciones globales y validación de entidades, junto con la clase `ClanBonusCalculator` para aplicar bonificadores de combate a los miembros en proximidad.

Minichat

Componente de interfaz de usuario (UI) en el cliente diseñado para gestionar la comunicación por texto. Emplea un diseño modular para integrarse fluidamente con el sistema de renderizado del HUD y el bucle de eventos, manteniendo el estado de entrada fuertemente encapsulado.

Funcionalidades:

- **Renderizado de historial:** mantenimiento y visualización dinámica del búfer de mensajes de chat en pantalla utilizando la clase `MiniChat`, con soporte para *scroll* y redimensionamiento.
- **Gestión de entrada de usuario:** captura y procesamiento asíncrono de eventos de teclado durante el modo de escritura a través del `EventHandler`.
- **Parseo de comandos:** integración del componente `ChatCommandParser` para evaluar cadenas de texto e identificar comandos de sistema y peticiones a NPCs, separándolos del canal de mensajería estándar.
- **Distribución de mensajes:** soporte en el servidor para mensajería privada y global gestionada a través del `ConnectionMonitor` y el `GameLoop` utilizando los DTOs de red `ChatDTO` y `PrivateChatDTO`.

Animaciones (entidades, armas y efectos)

Subsistema de presentación visual en el cliente responsable de traducir el estado lógico del servidor a secuencias gráficas interpoladas. Utiliza el patrón *Registry* para gestionar dependencias de texturas y delega las responsabilidades de dibujo para maximizar la cohesión y el rendimiento.

Funcionalidades:

- **Máquina de estados visual:** control de *sprite sheets* multidireccionales para el movimiento, inactividad y acciones de combate a través de `CharacterAnimator`, `CharacterSprites` y la capa de interpolación matemática `Motion`.
- **Renderizado de equipamiento dinámico:** actualización en tiempo real de armaduras, cascos, escudos y armas sobrepuestas al modelo base del personaje mediante el `EquipmentVisualRegistry` y la clase `EntityRenderer`.
- **Sistema de efectos (VFX):** arquitectura independiente para la instanciación y ciclo de vida de animaciones efímeras (impactos, curaciones, explosiones) orquestado por `FxSystem` y procesado visualmente por `FxAnimator`.

- **Transiciones críticas:** lógica visual específica para cambios de estado determinantes del modelo de dominio, como la secuencia gráfica de fantasmas manejada por las rutinas en *Death*.

Sistema de proyectiles

Arquitectura *end-to-end* distribuida para el modelado de mecánicas de ataque a distancia y magia. El servidor actúa como fuente de verdad física, mientras que el cliente interpola la representación visual independientemente de la tasa de *ticks* del motor central.

Funcionalidades:

- **Modelo físico de servidor:** gestión del ciclo de vida, cálculo cinemático de trayectorias (mediante velocidad y tiempo) y sistema de detección de colisiones continuo a través de *Projectile* y *ProjectileSystem*.
- **Resolución por estrategias:** utilización de los patrones *Strategy* (*IAttackDelivery*, *IHitEffect*) inyectados por el *WeaponFactory* para modularizar el daño de impacto simple o el daño de área (AoE) al alcanzar el objetivo.
- **Renderizado desacoplado:** instanciación y actualización del *sprite* en vuelo en el cliente mediante *ProjectileAnimator*, interpolando la posición visual basada en los vectores recibidos desde la red.

Protocolo de comunicación

Capa base de transporte y serialización de datos que establece la interfaz de comunicación cliente-servidor. Diseñada con un enfoque fuertemente tipado utilizando el patrón *Data Transfer Object* (DTO) para minimizar el *payload* y asegurar un flujo binario determinista.

Funcionalidades:

- **Serialización binaria:** empaquetado y desempaquetado de tipos primitivos directamente a nivel de *sockets*, optimizado en la clase *Protocol* asegurando la correcta conversión de *endianness* de red.
- **Diccionario de control:** mapeo estricto de eventos y directivas del juego utilizando la enumeración tipada *OpCodes*, garantizando un enrutamiento seguro y de complejidad $\mathcal{O}(1)$ para los paquetes entrantes.
- **Objetos de transferencia (DTOs):** segregación de interfaces mediante un conjunto de estructuras ligeras (como *SnapshotDTO*, *PlayerStatsDTO*, *LoginDTO*) para aislar la lógica de dominio de la carga útil enviada por la red.

2.3. Franco Bustos

Trabajó principalmente en la **capa de persistencia**, el **motor de fórmulas del juego**, el **sistema de spawn de criaturas** y la **generación procedural de mazmorras**. La implementación, distribuida entre el **servidor**, el **editor de mapas** y la **capa común**,

se enfocó en aislar las reglas de negocio de los detalles de infraestructura mediante estructuras de datos de tamaño fijo, inyección de configuración declarativa y la separación estricta entre lógica de dominio y representación en disco.

Persistencia

Capa de infraestructura responsable de la serialización y deserialización del estado completo del juego a disco. Se implementó un esquema **binario de tamaño fijo** con acceso directo (*seek + read/write*), evitando formatos de texto para minimizar la latencia y el uso de almacenamiento. La arquitectura separa la persistencia de jugadores de la persistencia del estado del mundo.

Funcionalidades:

- **Persistencia de jugadores:** almacenamiento binario de estado completo del jugador (posición, HP, maná, nivel, experiencia, oro, raza, clase, inventario completo y slots equipados) mediante la estructura empaquetada `PlayerPersistData` de 226 bytes validada con `static_assert`. La clase `PlayerDataStore` implementa un sistema de *indexado en memoria* con un archivo de índice separado (`players.idx`) que mapea `username` $\rightarrow$ `offset`, permitiendo lecturas por *seek* en $\mathcal{O}(1)$ y escrituras *append-only* para usuarios nuevos. El acceso concurrente se protege mediante un `std::mutex`.
- **Persistencia del mundo:** la clase `WorldDataStore` gestiona múltiples archivos binarios por mundo (`world_meta.bin`, `monsters.bin`, `npc_states.bin`, `ground_items.bin`, `clans.bin`, `bank_accounts.bin`), cada uno con un formato de cabecera + registros de tamaño fijo. Soporta la creación de múltiples mundos, listado de mundos guardados y carga selectiva de cada subsistema. Las estructuras `MonsterPersistData` (20 bytes), `NpcHeaderPersistData` (20 bytes), `GroundItemPersistData` (16 bytes) y `BankAccountHeaderPersistData` (12 bytes) siguen la misma filosofía de empaquetamiento con `#pragma pack(push, 1)` y validación estática de tamaño.
- **Serialización de entidades complejas:** para entidades con relaciones variables (NPCs con stock, clanes con miembros/pendientes/baneados, cuentas bancarias con slots), se utiliza un esquema de *header + payload dinámico*: la cabecera contiene un contador que determina cuántos registros hijos leer a continuación, permitiendo serializar grafos de datos sin recurrir a formatos autodescriptivos.

Mazmorras

Subsistema de generación procedural de contenido que define y aplica la estructura de mazmorras como *prefabs* reutilizables. La arquitectura separa la definición lógica del prefab (capa común) de su aplicación sobre el mapa (editor), siguiendo el patrón *Builder*.

Funcionalidades:

- **Definición del prefab:** la estructura `DungeonPrefab` define programáticamente una mazmorra de 16×22 tiles compuesta por una arena cuadrada cerrada con lava perimetral, un corredor de acceso de 2 tiles de ancho, decoraciones interiores (tumbas, agujeros, esqueletos) y botín de oro posicionado estratégicamente. La función

`buildPrefab()` construye el prefab como instancia *singleton* estática a través de `getDungeonPrefab()`.

- **Estampado en el mapa:** la clase `DungeonStamp` aplica el prefab sobre un `EditorMap`, modificando las capas de suelo, decoración, obstáculos e ítems. Implementa validaciones de colisión contra ciudades, otras mazmorras y bosques para prevenir superposiciones inválidas mediante la función `dungeonStampError()`.
- **Borrado reversible:** funcionalidad de limpieza completa (`clearDungeon`) que restaura las celdas originales (pasto), elimina decoraciones, obstáculos e ítems posicionados, y desregistra la mazmorra del modelo del editor. El borrado contextual (`eraseDungeonAt`) localiza automáticamente la mazmorra que contiene una celda dada y la elimina por completo.
- **Integración con el editor:** la función `dungeonOriginForClick()` calcula el origen de estampado a partir del click del usuario, centrando la mazmorra en la posición seleccionada.

Sistema de spawn de monstruos

Arquitectura de dos niveles para la gestión del ciclo de vida de criaturas en el mundo. El sistema se divide en **spawn por zonas** (criaturas regulares) y **spawn de bosses** (criaturas únicas de mazmorra), cada uno con su propia lógica de distribución, cooldowns y restricciones espaciales.

Funcionalidades:

- **Spawn por zonas (ZoneSpawnSystem):** sistema principal que gestiona la población de monstruos mediante regiones geográficas tipadas (`ZoneType`: `FOREST`, `DESERT`, `NORMAL`). Cada `SpawnZone` mantiene un contador de población actual vs. máxima, un *cooldown* configurable, y una lista de tipos de monstruos permitidos. La distribución inicial reparte la capacidad total equitativamente entre zonas, asignando el remanente a la zona normal. El algoritmo de posicionamiento (`getRandomSpawnPosition`) valida múltiples restricciones: evita zonas seguras, obstáculos, proximidad a NPCs y zonas de boss, garantizando que las criaturas de zonas específicas no aparezcan fuera de su territorio.
- **Spawn de bosses (BossSpawnSystem):** sistema independiente que gestiona criaturas únicas dentro de `BossZone`, áreas rectangulares con punto de spawn fijo y cooldown de reaparición configurable. Soporta cuatro tipos de boss (Balrog, Titán, Coloso, Aracne) seleccionados aleatoriamente. Implementa verificación de consistencia: si el `EntityManager` no contiene el boss registrado (por ejemplo, tras una desconexión), marca la muerte automáticamente e inicia el cooldown.
- **Modelo de zona (SpawnZone):** componente de bajo nivel que encapsula la geometría rectangular, el tipo de zona, la lista de monstruos permitidos y la máquina de estados del cooldown. Provee generación aleatoria de posiciones válidas con reinicios limitados y selección aleatoria de tipo de monstruo dentro de los permitidos.
- **DTO de spawn (SpawnRequest):** estructura ligera que encapsula el tipo de NPC, la posición y un puntero a la configuración del monstruo, utilizada como interfaz

desacoplada entre el sistema de spawn y el `World` para la instanciación efectiva de las entidades.

Razas y clases

Sistema de configuración declarativa que define los atributos base y modificadores de cada raza y clase de personaje del juego. Se implementó una separación completa entre datos y lógica, utilizando archivos TOML como fuente de verdad y un cargador tipado como puente hacia las estructuras de dominio.

Funcionalidades:

- **Modelo de datos:** las estructuras `RaceConfig` y `CharacterClassConfig` definen los factores multiplicativos que modifican las estadísticas base del jugador. Las razas (Humano, Elfo, Enano, Gnomo) ajustan vida, maná, fuerza, agilidad, inteligencia y recuperación. Las clases (Mago, Guerrero, Paladín, Clérigo) ajustan vida, maná, meditación y acceso a magia.
- **Carga tipada desde TOML:** la clase `CharacterConfigLoader` parsea el archivo `characters.toml` utilizando el helper genérico `TomlConfigHelper`, validando la existencia de todos los campos requeridos y produciendo un `CharacterConfigs` que agrupa la configuración del jugador base, las razas y las clases en `std::unordered_map` indexados por enumeración tipada (`Race`, `CharacterClass`).
- **Enumeraciones de dominio:** las enumeraciones `Race` y `CharacterClass` en `types.h` proveen tipado fuerte para evitar errores por cadenas literales, junto con funciones utilitarias `getRaceName()` y `getClassName()` para la representación textual en el protocolo y la UI.

World y orquestación del estado del juego

Nota: la clase `World`, al ser la entidad central y de mayor envergadura del servidor, fue modificada a lo largo de todo el desarrollo por la totalidad del equipo conforme se integraban nuevos subsistemas.

Clase principal del servidor que actúa como **fachada** y **orquestador** del estado completo de una partida. Implementa las interfaces `IWorldContext` (para consultas desacopladas del sistema de clanes) e `ICombatEventCallback` (para reaccionar a muertes de monstruos y jugadores), centralizando la coordinación entre todos los subsistemas sin asumir responsabilidad directa sobre su lógica interna. **Componentes internos:**

- **EntityManager:** registro centralizado de entidades activas, manteniendo colecciones de `Player`, `Monster` e `Interactable` (NPCs ciudadanos) con asignación incremental de `entityId` y mapeo bidireccional `dbId ↔ entityId` para resolución en $\mathcal{O}(1)$.
- **Map:** modelo espacial del mundo que centraliza toda la información topológica y las reglas de transitabilidad del mapa. Implementa una **arquitectura por capas** donde cada responsabilidad se delega a un componente independiente:

- **CollisionLayer** (doble instancia): una capa para obstáculos estáticos (paredes, lava, decoraciones bloqueantes) y otra para colisiones dinámicas de entidades (jugadores y monstruos). Ambas operan sobre una grilla booleana bidimensional e implementan detección de colisión puntual y *ray-casting* con el algoritmo de Bresenham para validar línea de visión entre dos posiciones.
- **GroundItemLayer**: capa de ítems en el suelo indexada por posición en un `unordered_map` con hash personalizado. Soporta un stack por tile (regla del juego original), colocación con búsqueda en espiral (radio máximo de 50 tiles) cuando la celda está ocupada, recolección atómica y serialización directa a `GroundItemPersistData`.
- **SafeZoneLayer**: gestión de zonas seguras nombradas (ciudades) donde se prohíbe el combate. Provee consulta posicional y resolución de nombre de zona.
- **NPCLayer**: registro posicional de NPCs ciudadanos (comerciantes, banqueros, sacerdotes) con búsqueda por rango para el sistema de interacción.

La clase `Map` también gestiona la carga completa del mapa desde archivos JSON, parseando zonas seguras, obstáculos, NPCs, monstruos, mazmorras, bosques, desiertos, zonas de boss e ítems iniciales. Expone consultas clave utilizadas por múltiples sistemas: `canMoveTo()` (combina ambas capas de colisión), `findClosestFreePosition()` (búsqueda en espiral para spawn de jugadores), `placeItemNearby()` (colocación inteligente de loot) e `isSafeZone()` (consultada por el sistema de combate y de spawn).

- **Subsistemas delegados**: `CombatSystem` (resolución de combate PvP y PvE), `ProjectileSystem` (proyectiles en vuelo), `ZoneSpawnSystem` y `BossSpawnSystem` (spawn de criaturas), `ResurrectionService` (resurrección diferida), `InteractionService` (diálogos con NPCs), `ClanController/ClanService/ClanRepository` (gestión de clanes) y `EventPublisher` (cola de eventos del mundo).
- **Configuración inyectada**: `ItemRegistry` (catálogo de ítems), `CharacterConfigs` (razas y clases), `InventoryConfig`, `GlobalBank` y `ServerConfig`, todos recibidos por referencia o valor en el constructor.

Funcionalidades:

- **Ciclo de actualización (update)**: ejecutado por el `GameLoop` a tasa constante, actualiza secuencialmente el estado de todos los jugadores, ejecuta la inteligencia artificial de monstruos (detección del jugador más cercano, persecución con *path-finding* por candidatos priorizados, ataque por cooldown), procesa resurrecciones completadas, limpia monstruos muertos liberando sus colisiones, y solicita nuevos spawns tanto de criaturas regulares como de bosses.
- **Inteligencia artificial de monstruos**: lógica de comportamiento integrada directamente en el ciclo de actualización. Cada monstruo busca al `Player` más cercano dentro de su rango de detección (`findNearestPlayer`), evitando jugadores muertos, en zona segura o fuera del área de boss correspondiente. El movimiento hacia el objetivo (`moveMonsterTowards`) genera una lista priorizada de posiciones candidatas (diagonal, ortogonal principal, laterales), seleccionando la primera que cumpla las restricciones de colisión y zona.

- **Gestión del ciclo de vida de jugadores:** `addPlayer()` integra la creación de jugadores nuevos con la restauración desde datos persistidos, aplicando la configuración de raza y clase, buscando posiciones libres cercanas al spawn o a la posición guardada, y notificando a los compañeros de clan. `removePlayer()` finaliza interacciones activas, notifica al clan y libera la colisión.
- **Callbacks de combate:** la implementación de `onMonsterDeath` orquesta la resolución de loot (delegando al `LootResolver` y al `ItemRegistry`), coloca el botín en el suelo a través del `Map`, emite mensajes de evento al asesino o broadcast para bosses, y notifica al sistema de spawn correspondiente. `onPlayerDeath` delega al flujo de penalización por muerte (pérdida de XP y oro excedente).
- **Generación de snapshots:** el método `generateSnapshot()` recopila el estado visual de monstruos, jugadores, ítems en el suelo y proyectiles activos en un `SnapshotDTO`, enviado por el `GameLoop` a todos los clientes conectados en cada tick.
- **Fachada de persistencia:** expone métodos `get*PersistData()` y `restore*()` para cada subsistema (monstruos, NPCs con stock, ítems en el suelo, clanes, cuentas bancarias), delegando la extracción y restauración del estado a cada componente y permitiendo que el `GameLoop` orqueste el guardado periódico (cada 30 segundos) a través de `WorldDataStore` y `PlayerDataStore`.

FormulaEngine

Motor centralizado de cálculos numéricos del juego implementado como *Singleton*. Encapsula todas las fórmulas de combate, progresión, economía y regeneración, aislando las reglas de balance del resto de la lógica de negocio y permitiendo su ajuste sin modificar los sistemas consumidores.

Funcionalidades:

- **Estadísticas de personaje:** cálculo de vida máxima ($\text{Constitución} \times \text{FClase} \times \text{FRaza} \times \text{Nivel}$) y maná máximo ($\text{Inteligencia} \times \text{FClase} \times \text{FRaza} \times \text{Nivel}$), parametrizados por los factores de raza y clase inyectados desde la configuración.
- **Sistema de combate:** daño base ($\text{Fuerza} \times \text{rand}(\text{Min}, \text{Max})$), mitigación por armadura agregada (suma de tiradas aleatorias de armadura, escudo y casco), evaluación exponencial ($\text{rand}(0,1)^{\text{Agilidad}} < 0.001$) y ataques críticos por probabilidad configurable.
- **Progresión y experiencia:** límite de nivel ($1000 \times \text{Nivel}^{1.8}$), ganancia de XP por ataque ($\text{Daño} \times \max(\text{NivelV\acute{ic}tima} - \text{NivelAtacante} + 10, 0)$), ganancia por kill ($\text{rand}(0, 0.1) \times \text{VidaMax} \times \max(0, 10 - \text{DifNivel})$), pérdida por muerte (25 % del límite), y clasificación de *newbie* ($\text{nivel} \leq 12$) con restricción de PvP por diferencia de niveles (≤ 10).
- **Economía:** límite de oro seguro ($100 \times \text{Nivel}^{1.1}$), cálculo de exceso perdido al morir y determinación del oro dropeado por NPCs ($\text{rand}(0, 0.2) \times \text{VidaMaxNPC}$).
- **Regeneración:** recuperación pasiva de vida proporcional al factor de raza y al tiempo transcurrido, y recuperación por meditación modulada por el factor de clase, la inteligencia y el tiempo.

- **Resolución de loot (LootResolver):** sistema complementario que utiliza `FormulaEngine` para determinar probabilísticamente las recompensas al morir un NPC: 80 % nada, 8 % oro, 1 % poción aleatoria, 1 % ítem aleatorio. Los bosses tienen loot garantizado (ítem único, oro fijo y botín extra) definido en su configuración.

ItemRegistry

Catálogo centralizado e inmutable de definiciones de ítems del juego, implementado con el patrón *Registry*. Actúa como fuente de verdad única para las propiedades de armas, armaduras y consumibles, eliminando la duplicación de datos y garantizando la consistencia a través de todos los sistemas que consultan información de ítems.

Funcionalidades:

- **Carga desde configuración:** el constructor consume los archivos TOML a través de `ItemConfigLoader`, instanciando cada ítem con su fábrica correspondiente (`WeaponFactory`, `ArmorFactory`, `ConsumableFactory`) y almacenándolo en mapas hash segmentados por tipo (`weapons`, `armors`, `consumables`). La detección de IDs y nombres duplicados se valida en tiempo de carga.
- **Consultas tipadas en $\mathcal{O}(1)$:** los métodos `get_weapon()`, `get_armor()` y `get_consumable()` devuelven punteros `const` al tipo concreto, permitiendo *downcasting* seguro sin necesidad de `dynamic_cast`. El método genérico `get_item()` realiza una búsqueda encadenada por todos los mapas.
- **Búsqueda por nombre:** el método `getItemByName()` implementa una búsqueda *case-insensitive* sobre todo el catálogo, utilizada para resolver comandos de texto del chat y del sistema de comercio.
- **Semántica de propiedad:** el registro mantiene `std::unique_ptr` para cada ítem, bloqueando copias del registro mediante `delete` del constructor de copia. Se provee semántica de movimiento para permitir la transferencia de propiedad durante la inicialización.
- **Consultas auxiliares:** métodos `isStackable()` (armas y armaduras no son apilables), `getPotionIds()` (filtra consumibles de tipo `HEALTH` y `MANA`) y `getAllDroppableItemIds()` (excluye el oro con ID 1), utilizados por el `LootResolver` y el sistema de inventario.
- **Jerarquía de ítems:** la clase base abstracta `Item` define la interfaz polimórfica (`isMagic()`, `is_wearable()`, `equip_on()`), extendida por `Weapon` (con estrategias de entrega y efecto de impacto inyectadas), `Armor` (con subclases `BodyArmor`, `Helmet`, `Shield`) y `Consumable` (con tipos `HEALTH`, `MANA`, `BOOST_STR`, `BOOST_AGI`).

2.4. Bruno Pezman

El trabajo realizado abarcó el diseño e implementación de componentes transversales a las tres capas del proyecto: infraestructura de build e instalación, modelo de entidades del servidor y sistemas de juego. Se trabajó desde la configuración del entorno de compilación y despliegue hasta la lógica central del servidor, incluyendo la jerarquía de ítems, la entidad `Player` con sus componentes, el sistema de combate y los NPCs interactivables

Sistema de build (CMakeLists.txt y Makefile)

Se configuró el sistema de build del proyecto en dos capas. El `CMakeLists.txt` raíz organiza el proyecto en módulos independientes (`client`, `server`, `editor`, `tests`), cada uno habilitables por separado, y gestiona automáticamente todas las dependencias de terceros via `FetchContent` (SDL2, GoogleTest, nlohmann_json, entre otras). Por encima, el `Makefile` actúa como interfaz de conveniencia que unifica en un solo comando las operaciones más frecuentes: compilar, correr tests, analizar memoria con Valgrind, lanzar los binarios y gestionar la instalación global.

Instalador y Desinstalador (installer.sh / uninstaller.sh)

Se desarrolló un script de instalación invocado por `make install` que automatiza el despliegue completo de la aplicación: instala dependencias del sistema, compila el proyecto, ejecuta los tests como condición de éxito, y copia binarios, assets y configuración a los directorios estándar del usuario. Los binarios se exponen globalmente mediante *wrapper scripts* que resuelven correctamente el directorio de trabajo, y se registran entradas `.desktop` para integración con el entorno gráfico.

El desinstalador (`make uninstall`) remueve los binarios del sistema, iconos, etc y pregunta sobre la persistencia de la base de datos y de la carpeta build antes de que sean también desinstaladas en caso de que no requieran serlo.

Jerarquía de ítems

Se diseñó la jerarquía de clases de ítems del servidor. La clase base abstracta `Item` define la interfaz común; `Armor` la extiende para equipamiento defensivo, del cual derivan `BodyArmor`, `Helmet` y `Shield` como clases finales. `Weapon` incorpora el patrón *Strategy* con dos ejes independientes: cómo viaja el ataque (`IAttackDelivery`) y qué ocurre al impactar (`IHitEffect`). `Consumable` cubre pociones y elixires con efectos inmediatos o temporales sobre el jugador. En todos los casos, la lógica de equipamiento se delega al propio ítem vía `equip_on()`, evitando `dynamic_cast` en el sitio de llamada.

Player y sus componentes

Se implementó `Player` como entidad central del servidor mediante composición: `StatsComponent` gestiona atributos, vida, maná y progresión; `InventoryComponent` administra la grilla de ítems y el oro; `EquipmentComponent` centraliza el equipamiento y el cálculo de defensa; `StateComponent` implementa una máquina de estados (*normal/ghost/meditating*) que filtra las acciones disponibles; y `RegenerationComponent` aplica recuperación pasiva tick a tick. `Player` implementa la interfaz `Attackable` y soporta persistencia y serialización para snapshots de red.

Sistema de combate

Se implementó `CombatSystem` como sistema centralizado que orquesta todos los combates del servidor, con puntos de entrada diferenciados para ataques de jugadores y de monstruos. Aplica bonificaciones de clan, notifica eventos y respeta reglas de *fair play* configurables. El comportamiento de cada arma queda completamente encapsulado en

sus estrategias: `IAttackDelivery` determina si el ataque es inmediato o viaja como proyectil, e `IHitEffect` determina si produce daño físico, daño mágico o curación.

NPCs e interacción

Se implementaron los tres tipos de NPC interactuables del juego (`Banker`, `Merchant`, `Priest`), todos bajo la interfaz `Interactable`. Cada NPC delega el manejo de comandos en handlers polimórficos indexados por tipo de comando, lo que permite extender el comportamiento de cada NPC sin modificar su clase.

3. Herramientas Utilizadas y Fuentes de Aprendizaje

El proyecto ha sido realizado en c++ bajo POSIX 2008, utilizando herramientas y tecnologías como:

- **Sistema Operativo:** Distribuciones Ubuntu 24.04 o Xubuntu 24.04.
- **Compilador:** GNU Compiler Collection (GCC) v2.
- **Lenguaje de Programación:** C++20.
- **Entorno de Desarrollo Integrado (IDE):** Visual Studio Code.
- **Bibliotecas Gráficas:** SDL2, SDL2pp y Qt6.
- **Framework de Pruebas:** GTest.
- **Herramienta de Chequeo de Memoria:** Valgrind.
- **Formato de Configuración:** TOML.
- **Control de Versiones:** Git.

4. Estado final de Entrega y Retrospectiva

Reflexión final

El desarrollo de Argentum Online nos permitió llevar a la práctica los conceptos del curso en un proyecto de escala real. Logramos cumplir con la consigna propuesta, entregando un sistema cliente-servidor funcional con editor de mapas, persistencia y múltiples mecánicas de juego.

El mayor desafío técnico fue SDL2: la gestión del ciclo de renderizado, las animaciones y la sincronización visual con el estado del servidor requirieron un esfuerzo considerable de investigación y ajuste iterativo. Superarlo nos dejó un conocimiento profundo de la biblioteca que no hubiéramos alcanzado de otra forma.

En cuanto al proceso de trabajo, la organización del equipo fue un factor determinante. La comunicación diaria y la proactividad de todos los integrantes permitieron que el desarrollo avanzara de forma sostenida y sin bloqueos prolongados. Establecimos desde el principio la regla de no mergear código roto, respaldada por una suite de tests unitarios que actuaba como red de seguridad antes de integrar cambios y el mantenimiento del

pre-commit para llevar un flujo limpio y no acumular errores en el tiempo. Esto redujo drásticamente los conflictos de merge y nos permitió construir sobre una base siempre estable.

Finalmente, mantener un archivo compartido de seguimiento de avance nos dio visibilidad colectiva sobre el estado real del proyecto en todo momento, facilitando la coordinación y evitando que el trabajo de un integrante bloqueara al resto. En conjunto, estas prácticas convirtieron un proyecto complejo en una experiencia de desarrollo ordenada y, sobre todo, disfrutable.